

Adaptive Control of a DC Motor

Felix Raffl, Lenard Wild

1 Introduction

Example 7.6 of the reader cites the experiment by Abram and Fieg, where the adaptive controller is implemented on an Arduino Due [1, 2]. The present report realises the same self-tuning pole-placement idea on an own test rig. The hardware consists of an ESP32, an L298N H-bridge, and a DC motor with a quadrature encoder. The controller is not hand-coded on the microcontroller; instead, the real-time code is generated automatically from the Simulink model.

The structure follows the indirect adaptive-control scheme of Figure 7.1 in the reader: the plant parameters are estimated recursively, the controller is redesigned from these estimates, and the resulting incremental PI algorithm is executed on the ESP32 input/output interface. Figure 1 shows the arrangement for the motor rig. After calibration by the one-revolution encoder test, the first-order self-tuning loop was run on the rig. The second-order controller was also run on the rig, but is reported only as diagnostic evidence for why full-order identification fails there; the working second-order design is a simulation result.

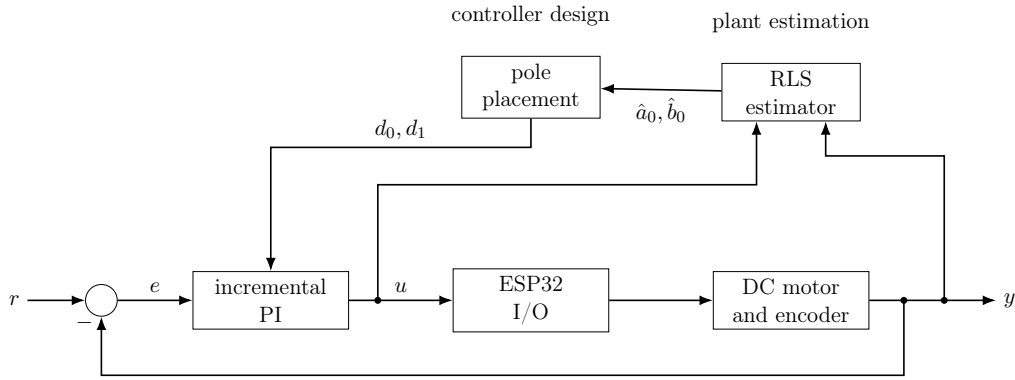


Figure 1: Self-tuning regulator used for the ESP32 DC motor rig.

2 First-order design

This mirrors the derivation of Example 7.6 of the reader. The first-order armature-to-speed relation is

$$J\dot{\omega} = -b\omega + K_m u. \quad (1)$$

Its continuous-time transfer function is

$$G(s) = \frac{\omega(s)}{u(s)} = \frac{K_m/J}{s + b/J}. \quad (2)$$

With zero-order hold sampling and $T = 0.08$ s, the discrete transfer function is written as

$$G(z) = \frac{b_0}{z + a_0}, \quad (3)$$

where

$$a_0 = -\exp\left(-\frac{bT}{J}\right), \quad b_0 = \frac{K_m}{b} \left(1 - \exp\left(-\frac{bT}{J}\right)\right). \quad (4)$$

The identified parameter vector is therefore

$$x_e = [a_0 \quad b_0]^T. \quad (5)$$

The discrete plant model in (3) gives the one-step regression

$$y(k) = C(k)x_e(k), \quad C(k) = [-y(k-1) \quad u(k-1)]. \quad (6)$$

The recursive least-squares update with forgetting factor α is

$$L(k) = \frac{P(k-1)C^T(k)}{C(k)P(k-1)C^T(k) + \alpha}, \quad (7)$$

$$x_e(k) = x_e(k-1) + L(k)(y(k) - C(k)x_e(k-1)), \quad (8)$$

$$P(k) = \frac{(I - L(k)C(k))P(k-1)}{\alpha}. \quad (9)$$

The initial values are

$$P(0) = 100I, \quad x_e(0) = [-0.5 \quad 1.0]^T. \quad (10)$$

The covariance update is suspended without excitation, which prevents covariance windup. The controller-design step uses a guard $|b_0| > 10^{-3}$, avoiding division by a nearly singular plant gain.

The desired closed-loop polynomial is chosen as

$$q(z) = z^2 - 0.5z + 0.06 = (z - 0.3)(z - 0.2). \quad (11)$$

For $T = 0.08$ s, these discrete poles correspond to continuous poles close to -15 s $^{-1}$ and -20 s $^{-1}$. The target settling time is therefore about 250 ms. Writing $q(z) = z^2 + q_1z + q_0$, the controller parameters are computed from the current plant estimates by

$$d_0 = \frac{q_0 + a_0}{b_0}, \quad d_1 = \frac{q_1 + 1 - a_0}{b_0}. \quad (12)$$

The implemented controller is the incremental PI law

$$u(k) = u(k-1) + d_1e(k) + d_0e(k-1), \quad (13)$$

where $e(k) = r(k) - y(k)$. The saturated command, not the unsaturated internal value, is fed back as $u(k-1)$ in (13); this is the anti-windup mechanism used in the implementation.

Writing the plant as $G(z) = b(z)/a(z)$ and the controller as $G_C(z) = d(z)/c(z)$, the closed-loop transfer function is

$$T(z) = \frac{b(z)d(z)}{q(z)}, \quad (14)$$

which is the form used throughout this report. The transient cannot be shaped by the pole locations alone, because the numerator of (14) contains the controller zero

$$z_0 = -\frac{d_0}{d_1}. \quad (15)$$

This is the same structural effect described in the reader as a phase error caused by the different numerator order. A one-degree-of-freedom controller provides no independent numerator design and cannot remove the zero without changing the controller structure.

3 Second-order design

With armature inductance L_a , armature resistance R_a , inertia J , viscous friction b , and motor constant K_m , the continuous-time second-order transfer function is

$$G(s) = \frac{K_m}{L_aJs^2 + (L_ab + R_aJ)s + (R_ab + K_m^2)}. \quad (16)$$

After zero-order-hold discretisation this is written as

$$G(z) = \frac{b_1 z + b_0}{z^2 + a_1 z + a_0}. \quad (17)$$

The adaptive estimator therefore has four parameters, with regression vector

$$\varphi(k) = [-y(k-1) \quad -y(k-2) \quad u(k-1) \quad u(k-2)]. \quad (18)$$

The numerical values used here,

$$K_m = 0.05, \quad J = 0.001 \text{ kg m}^2, \quad b = 0.0005, \quad L_a = 2 \text{ mH}, \quad R_a = 2 \Omega,$$

are representative parameters of a small brushed DC motor, not measurements of the 25GA371 rig motor. The conclusion does not depend on their exact values, only on the electrical time constant being far shorter than the sampling time. They give continuous poles -998.7 s^{-1} and -1.75 s^{-1} , so the electrical time constant is about 1 ms. Table 1 shows the exact ZOH discretisation at the hardware sampling time and at a sampling time that resolves the electrical pole.

Table 1: Exact ZOH parameters of the second-order motor model.

T	a_1	a_0	b_1	b_0	b_0/b_1	z -poles	
80 ms	-0.8692	0.000000	1.8467	0.0218	1.18 %	0.8692	0
2 ms	-1.1322	0.1352	0.0284	0.0148	52.3 %	0.9965	0.1357

Example 7.6 of the reader does not use the free $c(z)$ of the general Diophantine system; it forces $c(z) = z - 1$, i.e. a PI controller. The second-order design does the same thing one order higher. Pure pole placement fixes only the poles, not the static gain, and left a 17.1 % steady-state error in simulation. To remove this error, the design forces

$$c(z) = (z - 1)(z + c_0), \quad (19)$$

which gives $T(1) = 1$. The controller coefficients follow from the Diophantine identity $a(z)c(z) + b(z)d(z) = q(z)$ that underlies Equation 7.16 of the reader. The degree assignment is not the one of Equation 7.16, which pairs a controller of order $n - 1$ with a closed loop of order $2n - 1$: forcing the integrator factor into $c(z)$ raises both degrees by one, so $c(z)$ and $d(z)$ are of order n and $q(z)$ is of order $2n$. The resulting linear system is solved for the controller coefficients at every sampling instant.

The desired closed-loop polynomial is

$$q(z) = z(z - 0.1357)(z - 0.9608)^2. \quad (20)$$

The fast electrical pole is deliberately left in place in (20) because it is already much faster than the desired bandwidth. If instead

$$q(z) = (z - 0.9608)^4$$

is demanded, the design slows the electrical pole by a factor of 50. The controller must then invert the fast plant dynamics and becomes unstable itself, with its pole at $|z| = 2.66$, and behind a saturation limit such a controller necessarily diverges. The design rule is therefore to leave modes that are much faster than the desired bandwidth where they are.

The working design uses $T = 2 \text{ ms}$, with PRBS dither added to the manipulated variable. Clipping does not corrupt the estimate because the regressor uses the post-saturation command. The estimator is used without forgetting, $\alpha = 1$, because without persistent excitation $\alpha < 1$ makes P grow without bound.

3.1 Identifiability on the rig

The ARX model is structurally identifiable at both sampling times. At $T = 80$ ms, $A(z) = z(z - 0.8692)$ and $B(z) = 1.8467z + 0.0218$ are coprime; the ZOH numerator zero is at $-b_0/b_1 = -0.0118$, so it does not cancel the pole at the origin. The failure on the rig is practical, or statistical, identifiability.

Two effects pull against each other with T . Sensitivity to the electrical mode decays as $\exp(p_e T)$, so at $T = 80$ ms the b_0 term is only 1.18 % of the output, while the speed quantisation step $d_q = 2\pi/(T \text{ CPR})$ grows as $1/T$, with variance proportional to $1/T^2$. With the measured CPR = 44, no sampling time satisfies both requirements.

Table 2: Open-loop second-order identification under encoder quantisation.

setting	step [rad/s]	a_1 err	b_1 err	$ a_0$ err	b_0 err
Encoder resolution at $T = 80$ ms					
CPR = 44	1.785	−25.6 %	−0.06 %	0.194	1882 %
CPR = 704	0.112	−0.12 %	0.00 %	0.0009	9.1 %
exact	–	0.00 %	0.00 %	1.4×10^{-5}	0.13 %
Sampling time at CPR = 44					
$T = 80$ ms	1.785	−25.6 %	−0.06 %	0.194	1882 %
$T = 20$ ms	7.140	−49.8 %	−0.97 %	0.462	934 %
$T = 5$ ms	28.560	−43.5 %	−11.2 %	0.382	129 %
$T = 2$ ms	71.400	−100.0 %	−100.0 %	0.135	−100 %

Table 2 uses open-loop, full-order PRBS data precisely to remove closed-loop bias and poor excitation as explanations, leaving the sensor. The a_0 column is an absolute error because the true value at $T = 80$ ms is numerically zero, so a relative error would be meaningless. Quantisation also corrupts the regressors $y(k-1)$ and $y(k-2)$ in (18), an errors-in-variables problem that biases the estimate rather than merely adding variance. This is why the hardware estimates settle on wrong values instead of scattering about the right ones.

At $T = 2$ ms, CPR = 44 gives only about 1.3 encoder counts per sample at a moderate 90 rad/s, and 10.9 counts per sample at the measured full-duty 780 rad/s, so the speed signal degenerates towards a one-bit signal. This is pure arithmetic from two measured constants and holds regardless of the motor model.

The mechanical pair is robust: b_1 is still recovered to 0.06 % at CPR = 44 and $T = 80$ ms. That is why the first-order design of Example 7.6 runs on this hardware and the second-order extension does not. The controller only needs the well-determined combinations of the parameters, not their individual values, so the closed loop stays stable. A stable closed loop is therefore not evidence of successful identification.

4 Simulation results

4.1 Adaptation to a change of the plant

A self-tuning regulator is only worth its cost if it holds the closed-loop poles when the plant itself changes. This is simulated in `simulink/adaptive_dcmotor_sim.slx`, which runs the first-order design of Section 2 at $T = 80$ ms with $\alpha = 0.98$ and the command clamped to $u \in [-255, 255]$. The inertia of the plant is raised by a factor of 24 at $t = 32$ s and restored at $t = 64$ s. That factor is the ratio of the two flywheel discs. The discs are not fitted to the rig, so the inertia change is a simulation result and is not claimed as a measurement.

Figure 2 shows the run. The estimate of the input gain follows both steps: b_0 falls when the inertia is raised, because $b_0 = (K_m/b)(1 - \exp(-bT/J))$ shrinks as J grows, and it recovers when the inertia is restored. The covariance trace holds at its initial value of 200 while the motor is

idle and collapses only once excitation begins, so the excitation gate does what it was built for: it prevents the windup that $\alpha < 1$ would otherwise produce in the absence of persistent excitation.

The run also shows where the design ends. With the inertia raised by 24 the actuator saturates, the command sits at ± 255 , and the speed can no longer follow the square-wave reference; it ramps between the levels instead of stepping. At the downward transition the loop is briefly driven by controller coefficients that still belong to the heavy plant, which produces an oscillation burst that decays within a few seconds as the estimator reconverges. The limit is set by the actuator, not by the estimator: no choice of controller coefficients lets a saturated actuator accelerate a load 24 times heavier.

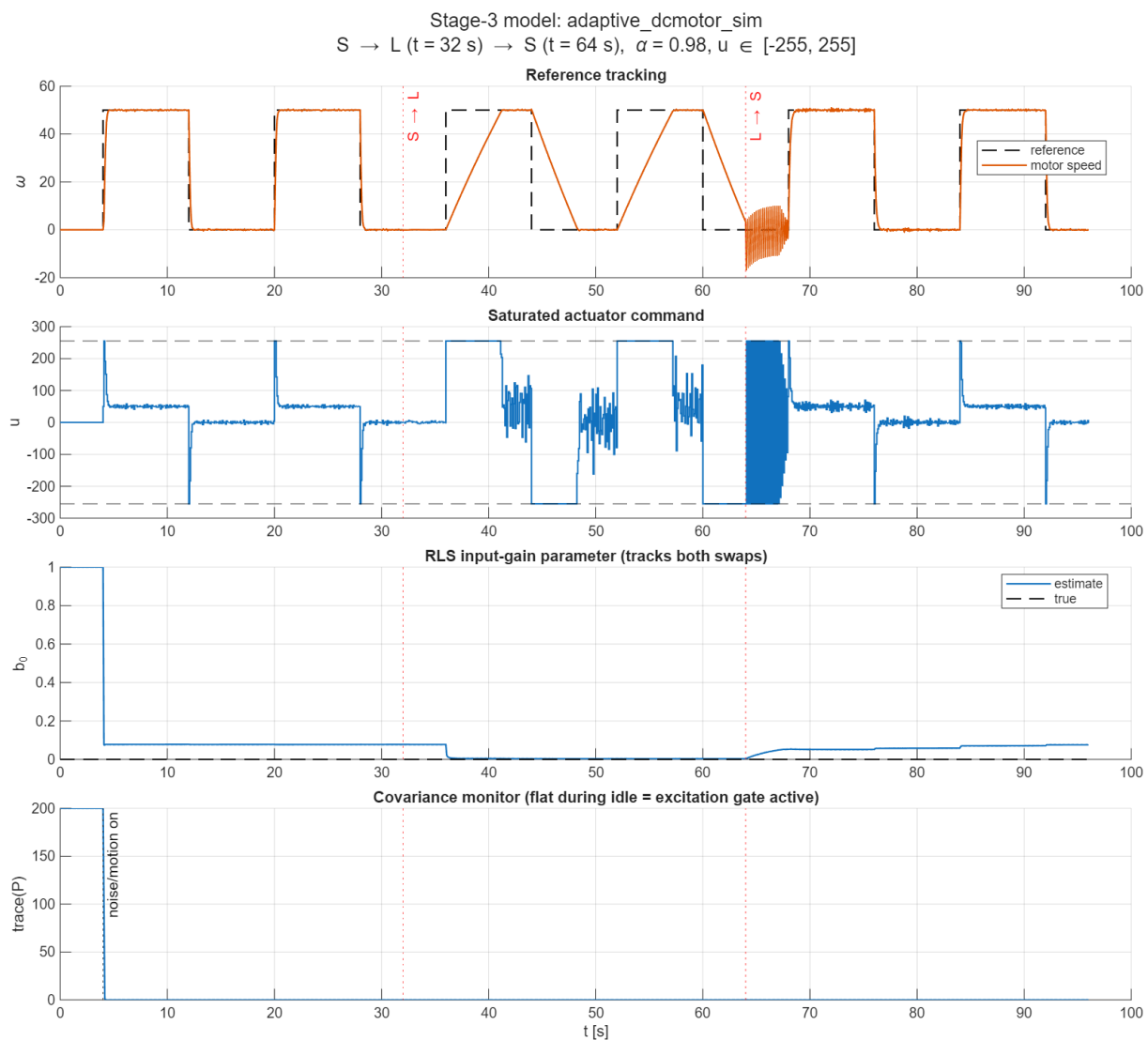


Figure 2: First-order adaptive loop against a plant whose inertia is raised by a factor of 24 at $t = 32$ s and restored at $t = 64$ s. Simulation only; the flywheels are not fitted to the rig.

4.2 Second-order identification

The validation run uses the second-order design with $T = 2$ ms. Its speed signal is exact and unquantised; the identifiability study in Section 3.1, Table 2, shows what changes with the rig encoder. Table 3 shows that all four parameters converge to their true ZOH values. In the window $t = 3$ s to 4 s, all four parameters lie within 0.65 % relative error of their true ZOH values.

The static closed-loop gain is $T(1) = 1.000$. For the step at $t = 5.5$ s, the response has no

Table 3: Validated second-order parameter estimates at $T = 2$ ms.

Parameter	Estimated, $t = 3$ s to 4 s	True	Absolute error
a_1	-1.131310	-1.132176	8.66×10^{-4}
a_0	0.134337	0.135200	8.63×10^{-4}
b_1	0.028348	0.028362	1.49×10^{-5}
b_0	0.014889	0.014832	5.71×10^{-5}

overshoot and the 2% settling time is 0.384 s. The trace of P remains bounded at 400. Figure 3 shows the output, command, parameter estimates, and covariance trace from this validation run.

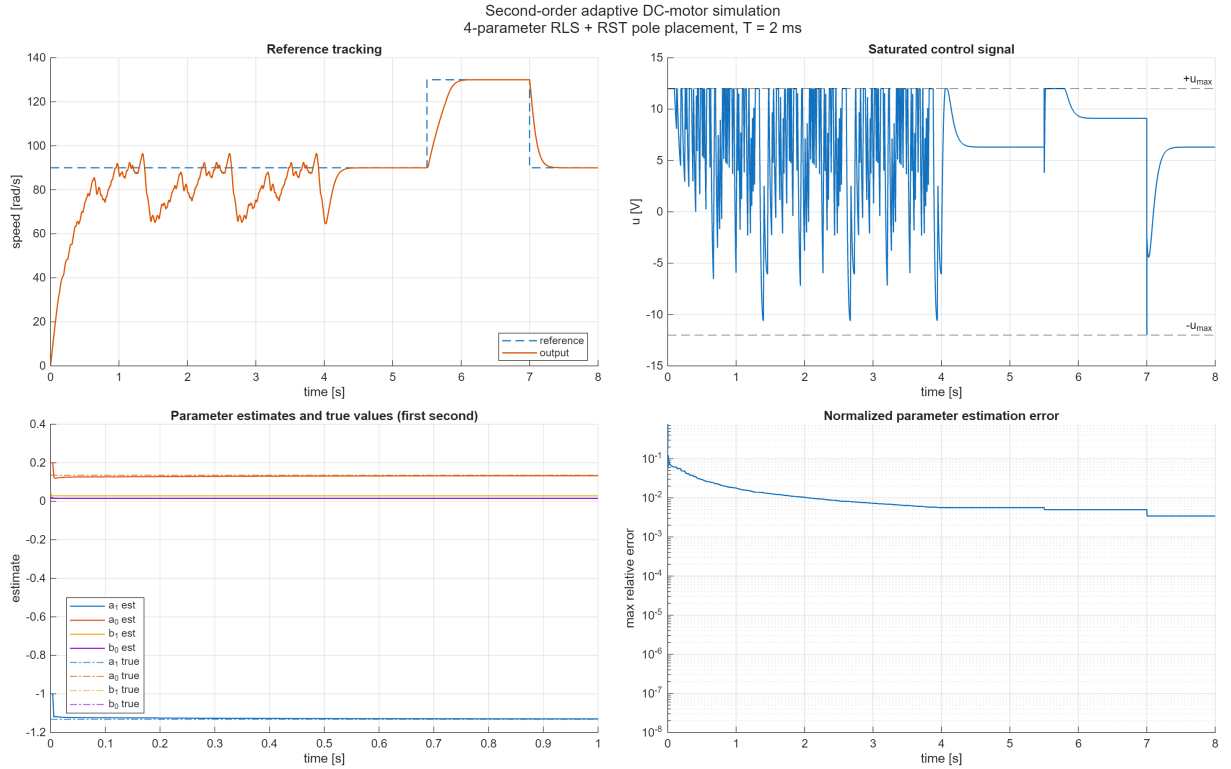


Figure 3: Validation run of the second-order adaptive RST controller.

5 Hardware realisation

The hardware platform is an ESP32-WROOM module connected to an L298N H-bridge. The L298N introduces an approximate voltage drop of 4 V, so with a 12 V supply the motor sees about 8 V. The motor is equipped with a quadrature encoder. The sampled controller uses the period $T = 0.08$ s, $P(0) = 100I$, $x_e(0) = [-0.5 \ 1.0]^T$, forgetting factor $\alpha = 0.98$, desired polynomial $q(z) = z^2 - 0.5z + 0.06$, and a command saturated at ± 255 . `simulink/adaptive_dcmotor.slx` is the first-order loop that runs on the rig, while `simulink/adaptive_dcmotor_2nd_hw.slx` is the second-order diagnostic run reported in Subsection 5.1.

The controller is implemented by automatic code generation from Simulink using the Embedded Real-Time (ERT) target, the ESP32 target support, and External Mode for Monitor and Tune operation. Table 4 gives the pin assignment.

The encoder resolution was determined by a one-revolution test as 44 counts per revolution, corresponding to 11 encoder impulses with x4 quadrature edge decoding. The speed conversion

$$\omega = \frac{2\pi\Delta N}{T \text{CPR}}$$

Table 4: ESP32 pin assignment for the DC motor test rig.

Signal	ESP32 pin
PWM	GPIO 14
IN1	GPIO 26
IN2	GPIO 27
Encoder A	GPIO 32
Encoder B	GPIO 33



Figure 4: Assembled DC motor rig. The motor, driver and controller are enclosed; the motor shaft protrudes on the right. The complete wiring diagram is included with the accompanying repository.

is therefore calibrated for $\text{CPR} = 44$. The measured full-duty speed of approximately 780 rad/s corresponds to about 7450 rpm , consistent with the gearbox-free 25GA371 speed range of roughly 3000 rpm to 8000 rpm and independently supporting the encoder scaling. Figure 4 shows the assembled rig.

The encoder arithmetic in Table 5 follows only from the two measured constants $\text{CPR} = 44$ and the full-duty speed 780 rad/s ; it is independent of any motor model. At $T = 2 \text{ ms}$ a moderate 90 rad/s gives about 1.3 counts per sample, so the speed signal degenerates towards a one-bit signal.

Table 5: Encoder resolution from the measured CPR and full-duty speed.

T	$2\pi/(T \text{ CPR}) \text{ [rad/s]}$	counts/sample at 780 rad/s	counts/sample at 90 rad/s
80 ms	1.78	437	50.4
20 ms	7.14	109	12.6
5 ms	28.56	27.3	3.15
2 ms	71.40	10.9	1.26

5.1 Measurements

The recorded hardware measurement is a closed-loop run of the second-order adaptive controller from `simulink/adaptive_dcmotor_2nd_hw.slx`. It uses $T = 80 \text{ ms}$, $\alpha = 0.98$, encoder scaling $2\pi/(0.08 \cdot 44)$, a square-wave reference between 0 and 780 rad/s with edges every 4 s, and a run length of 60 s.

The response in Figure 5 reaches and holds the upper setpoint. That setpoint is the measured full-duty speed, so the actuator has little headroom there. On falling edges the speed shows a limit cycle about standstill: the driver cannot brake actively and the motor only coasts down. The estimates in Figure 6 never settle. They jump at every reference edge and drift between edges; the estimate of the coefficient multiplying $y(k-2)$, the curve labelled a_2 in the figure and

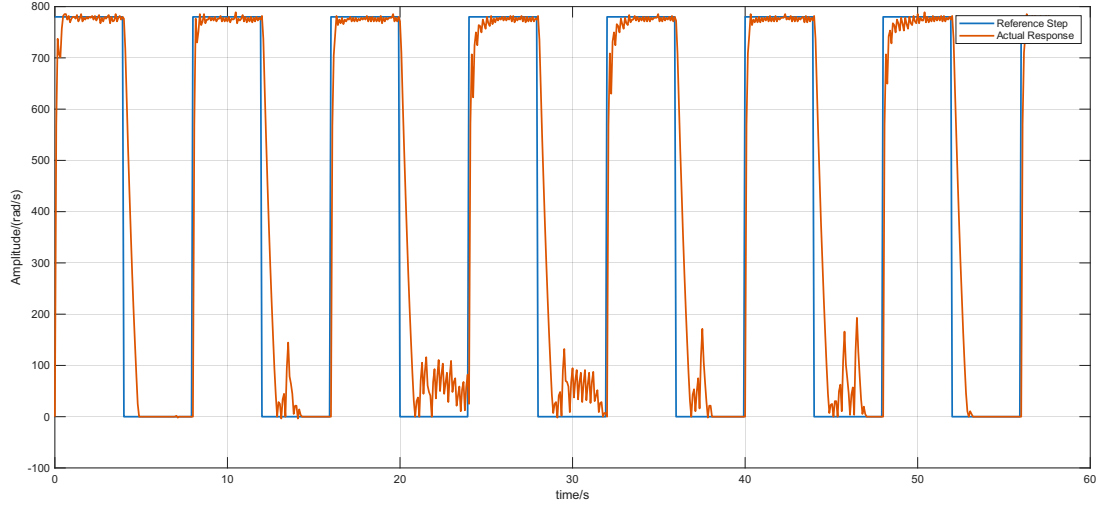


Figure 5: Closed-loop response of the second-order adaptive controller on the hardware rig.

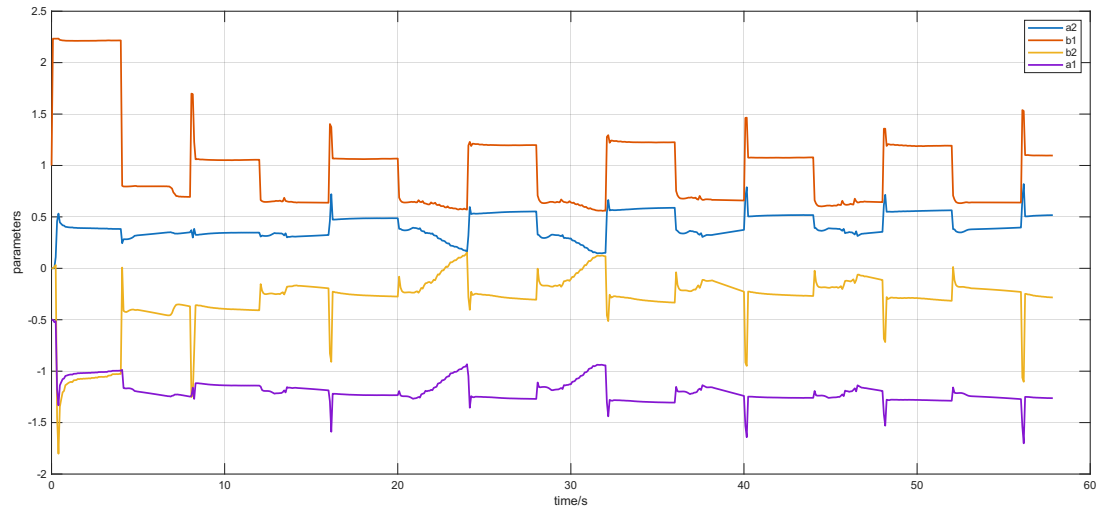


Figure 6: Second-order parameter estimates during the hardware run. The curves labelled a_1 and a_2 in the plot are the coefficients of $y(k-1)$ and $y(k-2)$, i.e. a_1 and a_0 in the notation of this report; the curves labelled b_1 and b_2 are the coefficients of $u(k-1)$ and $u(k-2)$, i.e. b_1 and b_0 .

a_0 here, hovers around +0.45, whereas the exact ZOH value at $T = 80$ ms is numerically zero in Table 1. This is the behaviour predicted by the identifiability study in Section 3.1, Table 2.

Three mechanisms act here. The decisive one is encoder quantisation acting on an ill-conditioned regression: that study isolates it with open-loop, full-order PRBS data and no forgetting, so poor excitation and forgetting are absent and the identification still fails. The excitation is also weak, because the reference is a square wave and, between edges, u and y are nearly constant, so information enters mainly at the edges. Finally, with $\alpha = 0.98$ at $T = 80$ ms, the memory horizon is $1/(1 - \alpha) = 50$ samples, or 4 s, exactly the interval between reference edges; what is learnt at one edge is therefore forgotten before the next. This run is also a closed-loop identification, for which least squares is biased in general; the open-loop study avoids that objection.

6 Conclusion

The first-order self-tuning regulator of Example 7.6 is implemented on the ESP32 rig, with the encoder scaling fixed by the one-revolution test at $\text{CPR} = 44$. The study explains why this is the order that can work there: the mechanical gain b_1 is recovered to 0.06 % even at

that resolution. The same derivation carried one order higher identifies all four parameters in simulation at $T = 2\text{ms}$ with an exact speed signal. On the rig the second-order model is structurally identifiable but not practically identifiable. At $T = 80\text{ms}$ the electrical mode contributes about 1 % of the output and is lost in the encoder quantisation; recovering it at that sampling time would need about 700 counts per revolution instead of 44. Shortening the sampling time restores the sensitivity to the electrical mode, but the speed resolution degrades as $1/T$, so with the measured encoder no sampling time satisfies both requirements. The second-order design therefore remains a simulation result, and the stable closed loop measured on the rig must not be mistaken for a successful identification.

References

- [1] A. Ravazzolo-Mehrle, *Advanced Control Engineering 2 - Optimal Control*. MCI Innsbruck, 2026. Reader, spring term 2026.
- [2] Abram and Fieg, “Adaptive control experiment with an arduino due.” As cited in the Advanced Control Engineering 2 reader.